

Blender Game Engine (BGE)

O BGE é uma das partes mais interessantes do *Blender* que permite a criação de jogos sem quase ter que recorrer à programação pois utiliza blocos lógicos. Esses blocos lógicos permitem controlar o movimento e o *display* de objetos, entre outros, dividindo-se em:

- **Sensores** – detetam eventos (por exemplo, tecla usada ou movimento do rato);
- **Controladores** – combinam as entradas de forma a ativarem uma resposta;
- **Atuadores** – iniciam tarefa ao receberem uma resposta positiva dos controladores.

- Abrir o Blender;
- Mudar o **Engine** para **Blender Game** e o *layout* para **Game Logic**;
- Alterar o cubo para a posição (5, 5, 0.5), com dimensão (1, 1, 1) e com uma cor difusa azul.

1. Navegação controlada

- Com o cubo selecionado, pressionar o botão **Add Sensor** e escolher o sensor **Keyboard**;
- Pressionar o **botão esquerdo do rato** sobre o campo vazio situado em frente da área **Key** e, depois, pressionar a **Tecla →** (esta tecla ficará atribuída a este sensor e quando pressionada ativará uma resposta positiva);
- Alterar o nome do sensor para “MovDir”;
- Pressionar o botão **Add Controller**, escolher o controlador **And**;
- Alterar o nome do controlador para “E_MovDir”;
- Pressionar o botão **Add Actuators** e escolher o atuador **Motion**;
- Alterar o valor do eixo dos XX, da área **Loc**, para 0.1 (o cubo irá saltar 0.1 unidades de Blender, cada vez que a Tecla → for usada);
- Alterar o nome do atuador para “Mov +X”;
- Ligar os pontos entre os blocos (a partir de um ponto, pressionar o **botão esquerdo do rato** e, mantendo esse botão pressionado, arrastar o rato até ao outro ponto);
- Na janela das propriedades, ícone **Render**, pressionar no botão **Start**, do painel **Embedded Player** (ou a **Tecla P**, com o cursor sobre a janela 3D View), e verificar que ao pressionar a Tecla → o cubo move-se para a direita;
- Retornar ao editor lógico, pressionando a **Tecla ESCAPE**;
- Fazer os três restantes movimentos para o cubo, controlados pela **Tecla ← (Loc X=-0.1)**, **Tecla ↑ (Rot Z=2°)** e **Tecla ↓ (Rot Z=-2°)**.

2. Navegação automática realista

- Selecionar a câmara e mudar para a vista da câmara;
- Posicionar a câmara em (13, -15, 10), rodada (60°, 0°, 35°);
- No ícone **Object Data**, painel **Lens**, alterar a **Focal Length** para 25;
- Adicionar um plano, de cor difusa verde, em (0, 0, 0.1), com dimensão 16x16;
- Aplicar ao plano 3 subdivisões, removendo as faces conforme a figura1;

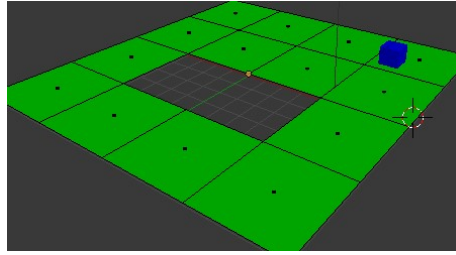


Figura 1: Características do plano 3D a modelar.

- Adicionar um cone na posição $(0, -7, 0.5)$ e com dimensão $(1, 1, 1)$;
- Com o cone selecionado, premir o botão **Add Sensor** e escolher o sensor **Always**;
- Pressionar o botão **Add Actuators** e escolher o atuador **Steering**;
- Colocar no campo **Target Object** o nome do cubo;
- Deseleccionar o botão **Lock Z Velocity**;
- Ligar os blocos e verificar (**Tecla P**) que o cone vai ter com o cubo, passando por cima do buraco do plano do chão como se ele não existisse;
- Selecionar o cone e no painel **Physics**, ícone **Physics** (da janela de propriedades), alterar o tipo da física para **Dynamic**;
- Verificar (**Tecla P**) que o cone cai no buraco do plano do chão;

- Para que o cone contorne os obstáculos deve-se recorrer a alguma inteligência artificial. Para isso, deve-se usar a técnica da malha de navegação. Para que os elementos que vão usar esta malha se movimentem automaticamente, devem estar localizados dentro dela. Assim sendo, fazer:

- Selecionar o plano e no painel **Navigation Mesh**, ícone **Scene** (da janela de propriedades), pressionar o botão **Build Navigation Mesh**;
- Selecionar o cone e, no atuador **Steering**, colocar o campo **Behavior** com o valor **Path Following** e o campo **Navigation Mesh** com o nome da malha de navegação criada;
- Verificar (**Tecla P**) que o cone procura o cubo sem cair no buraco;
- No modo de edição, selecionar uma aresta do plano (canto superior direito) e fazer a extrusão no eixo dos XX;
- Subir a aresta no eixo dos ZZ e fazer nova extrusão no eixo do XX;
- No modo objeto, colocar o cubo em cima da rampa construída;
- Verificar (**Tecla P**) que o cone não sobe a rampa, pois a malha de navegação não existe lá. Assim, apagar a **Navigation Mesh** existente;
- Selecionar o plano e no painel **Navigation Mesh**, ícone **Scene** (da janela de propriedades), alterar o campo **Max Slope** para 75° ;
- Voltar a pressionar o botão **Build Navigation Mesh** e verificar que a malha de navegação agora já sobe a rampa;
- Selecionar o cone e, no atuador **Steering**, colocar o campo **Navigation Mesh** com o nome da última malha de navegação que foi criada;
- Verificar (**Tecla P**) que o cone já consegue subir a rampa, apesar desta subida não ser muito correta (o cone entra para dentro da rampa);
- Selecionar o cone e, no atuador **Steering**, ativar o campo **N**;
- Na janela das propriedades, ícone **Render**, pressionar o botão **Start**, do painel **Standalone Player** e verificar que o cone já sobe a rampa;
- Retornar ao editor lógico, pressionando a **Tecla ESCAPE**.

3. Brincar com colisões

- Selecionar o cubo azul, colocar-lhe o tipo de física em **Dynamic** e ver (**Tecla P**) que se for posto no meio da rampa ele desce sozinho e é empurrado para fora do plano pelo cone;
- Adicionar um cubo, com dimensão (1, 1, 1), na posição (-7, 5, 0.5);
- Verificar, no painel **Physics**, ícone **Physics** (da janela de propriedades), que o cubo é do tipo **Static** e mudar o tipo do cubo azul para **Rigid Body**;
- Ver (**Tecla P**) que por causa do tipo de física que tem, nenhum elemento poderá tirar o cubo acabado de criar da sua posição, nem atravessá-lo, podendo-se assim criar limites no jogo, e verificar também que o comportamento do cubo azul é diferente;
- Adicionar uma esfera vermelha, com dimensão (1, 1, 1), na posição (2, 5, 0.5);
- Alterar o tipo de física da esfera para **No Collision**;
- Ver (**Tecla P**) que o cubo azul atravessa a esfera como se esta não existisse;
- Alterar agora o tipo de física da esfera e do cubo azul para **Dynamic**;
- Verificar (**Tecla P**) que o cubo azul já não atravessa a esfera, apenas a empurra;
- Com a esfera selecionada, premir **Add Sensor** e escolher o sensor **Collision**;
- Clicar no campo **M/P** e, na área logo em frente, selecionar o material azul. Isto faz com que apenas os elementos com o material igual ao escolhido nesta área, quando tocarem nesta esfera, ativem uma resposta positiva;
- Pressionar o botão **Add Actuators** e escolher o atuador **Edit Object**;
- Nesse atuador, alterar a opção para **End Object**;
- Ligar os blocos e ver (**Tecla P**) que a esfera desaparece quando o azul lhe toca;
- Copiar a esfera (**Teclas SHIFT + D**) e colocá-la noutra posição com o mesmo **XX**;
- Verificar (**Tecla P**) que o cubo azul faz com que ambas as esferas desapareçam, provando assim que a cópia de um elemento já associado a blocos lógicos, copia também essa parte.

4. Propriedades

- As propriedades são uma espécie de variáveis que guardam informação que se pode usar durante o jogo. No BGE existem dois tipos de propriedades: as básicas (criadas quando se prime o botão **Add Game Property**) e as de texto (criadas quando se prime o botão **Add Text Game Property**);
- A única diferença entre as propriedades básicas e as de texto é que as últimas permitem mostrar no ecrã a informação que contêm;
- Um exemplo de uso das propriedades de texto pode ser o seguinte:
 - Criar o texto "0" (apenas o carácter zero), alinhado com a vista, dar-lhe o nome "Contador" e colocá-lo no canto superior direito da imagem;
 - Com o texto selecionado, no painel **Properties**, do editor lógico, pressionar o botão **Add Text Game Property**;
 - Alterar o tipo da propriedade **Text** para **Integer**;
 - Juntamente com o elemento de texto, selecionar, também, uma das esferas vermelhas (**SHIFT + Botão direito do rato**);
 - Adicionar um atuador **Property** ao elemento Contador (Texto);
 - Mudar o campo **Mode** para **Add**;

- Colocar no campo **Property**, o nome da propriedade **Text Game Property** (a que se associou ao texto anteriormente criado);
- Colocar no campo **Value** o valor 1 (vai adicionar esse valor ao *Contador* que anteriormente foi inicializado a zero);
- Ligar o controlador **And** da esfera vermelha (ligado ao sensor *Collision*) ao atuador **Property** do elemento de texto;
- Verificar (**Tecla P**) que o contador no canto superior direito da imagem é incrementado quando o cubo azul toca na esfera que se programou;
- Juntar aos elementos selecionados a outra esfera e ligar o seu controlador **And** ao atuador **Property** do elemento de texto;
- Verificar (**Tecla P**) que o contador vai agora até 2.

- Um exemplo de uso das propriedades básicas pode ser o seguinte:

- Adicionar ao jogo um plano com tamanho 30x30, nome “Test” e na posição (0, 0, -2);
- No painel **Properties**, do editor lógico, pressionar o botão **Add Game Property**;
- Alterar o nome da propriedade para “vida”, pôr o tipo da propriedade como **Integer** e alterar o valor da propriedade para 1;
- Pressionar no botão  (para permitir que depois se possa ver o comportamento desta propriedade no modo de debug);
- Com o plano selecionado, adicionar um sensor **Collision**;
- Adicionar um atuador **Property**;
- Mudar o campo **Mode** para **Add**;
- Colocar no campo **Property** a propriedade “vida”;
- Colocar no campo **Value** o valor -1 (vai somar esse valor ao que a propriedade *vida* tem, ou seja, subtrair-lhe 1);
- Ligar os blocos;
- Na janela **Info**, escolher a opção **Game** e ativar o campo **Show Debug Properties**;
- Verificar (**Tecla P**) que quando o cubo cai do plano de jogo e toca no plano de baixo, a propriedade “vida” passa a 0;
- Com o plano selecionado, adicionar um sensor **Property**;
- Colocar no campo **Property** a propriedade “vida”;
- Colocar no campo **Value** o valor 0 (vai ativar uma resposta positiva quando a propriedade “vida” for igual a 0);
- Adicionar um atuador **Game**;
- Alterar o campo **Game** para **Quit Game**;
- Ligar os blocos;
- Verificar (**Tecla P**) que quando a propriedade “vida” fica com o valor de 0, o jogo termina.

- O plano *Test* serve apenas para otimizar o jogo (não esperar muito enquanto o cubo cai do plano de jogo) e por isso não deve ser visto no jogo. Assim:

- Selecionar o plano;
- No painel **Physics**, ícone **Physics** (da janela de propriedades), pressionar o botão **Invisible**;
- Verificar (**Tecla P**) que o plano já não é renderizado.

5. Utilização de diferentes câmaras no jogo

- Os jogos costumam ter várias câmaras para mostrar os cenários de pontos de vista diferentes. Um exemplo de como fazer isso é o seguinte:

- Adicionar uma câmara, de nome *CamGo*, na mesma posição que o cubo azul e rodada ($90^\circ, 0^\circ, 90^\circ$);
- Selecionar esta câmara e depois o cubo azul, e criar uma relação de parentesco (**Teclas CTRL + P**, seguido da opção **Object**), para que quando o cubo se move a câmara o faça de igual forma – na criação destas relações escolhe-se sempre o filho em primeiro lugar e só depois o pai;
- Selecionar apenas a câmara e adicionar um sensor **Mouse**;
- Mudar o campo **Mouse Event** para **Right Button**;
- Adicionar um atuador **Scene** e mudar o campo **Mode** para **Set Camera**;
- Colocar no campo **Camera Object** o nome da câmara adicionada;
- Ligar os blocos e ver (**Tecla P**) que quando o botão direito do rato for pressionado a vista muda;
- Fazer o mesmo para que o botão esquerdo do rato reponha o uso da câmara original.

6. Brincar com o som

- Os jogos costumam ter sons que os tornam mais interessantes. Um exemplo de como fazer isso é o seguinte:

- Selecionar o cubo azul e criar uma *Game Property*, chamada **toque** (não interessa, para o caso, o tipo e o valor);
- Adicionar um elemento **empty**, do tipo **Plain Axes**, na mesma posição que uma das esferas, (2, 5, 0.5). Apesar deste elemento nunca se ver no jogo, é muito usado para marcar localizações importantes para o mesmo;
- Adicionar um sensor **Near**;
- No campo **Property**, colocar **toque**;
- Deixar o campo **Distance** a 1 (medida dos elementos do jogo e distância a partir da qual o sensor enviará o sinal positivo);
- Colocar 1000 no campo **Reset Distance** (este campo permite que o som possa voltar a ser tocado quando o cubo se afastar do **empty** 1000 unidades do Blender. Como se pretende que a campainha não volte a tocar, coloca-se um valor que seja impossível o cubo percorrer. Existem outras formas de fazer isto, mas esta é a mais simples);
- Adicionar um atuador **Sound**, pressionar o botão **Open**, ativar o botão **Cache** (lado esquerdo do ecrã) e selecionar o ficheiro “*campainha.wav*” (o BGE trabalha melhor com ficheiros wave);
- Ligar os módulos;
- Verificar que quando o cubo colide com a esfera ouve-se o som escolhido. Se fosse pretendido que o som tocasse quando cada esfera fosse “comida”, teria que se repetir os passos anteriores para cada uma delas.

7. Screen Menu

- Os jogos costumam começar com um menu principal e só depois vão para o cenário do jogo, quando pressionado um botão ou uma tecla. É chamado de *Screen Menu*. Um menu deste género pode ser feito da seguinte forma:

- Criar uma nova cena, chamada “*MenuIntro*” (botão **+**, seguido da opção **New**, da área das cenas, da janela *Info*);
- Mudar o **Engine** para **Blender Game**;
- Adicionar à cena uma câmara, posicionada em (10, -10, 7) e rodada (60°, 0°, 35°);
- Mudar para a vista da câmara;
- Adicionar à cena um plano, na posição (2, 1, 0) e rodado (60°, 0°, 35°);
- Com o plano selecionado, adicionar um sensor **Mouse**;
- Mudar o campo **Mouse Event** para **Mouse Over** (ativa resposta positiva quando o rato passar por cima do plano);
- Adicionar um atuador **Scene**;
- Pôr o campo **Mode** a **Set Scene**;
- Colocar **Scene** no campo **Scene**;
- Ligar os blocos lógicos;
- Ver (**Tecla P**) que não aparece o cursor do rato;
- No painel **Display**, ícone **Render** (da janela de propriedades), ativar a opção **Mouse Cursor**;
- Ver (**Tecla P**) que o cursor já aparece e que quando passa na zona do plano, a cena é mudada;
- Como esta não é a forma habitual nos *screen menus*, com o plano selecionado, adicionar um sensor **Mouse**;
- Ligá-lo ao atuador **AND** que tem ligado o outro sensor **Mouse**;
- Verificar (**Tecla P**) que para a cena mudar o cursor do rato tem que estar sobre o plano e o botão esquerdo do rato tem que ser pressionado.

8. Atuador Action

- É possível usar qualquer tipo de animação (feitas com *keyframes*, *shapekeys* ou baseadas em armaduras) nos jogos. Um exemplo disto pode ser feito da seguinte maneira:

- Mudar o *layout* para **Animation** e escolher a cena *MenuIntro*;
- Colocar a vista da câmara e selecionar o plano;
- Colocar a *frame* da **Timeline** a 1;
- Criar uma *keyframe* de redimensionamento (**Tecla I**, seguida da opção **Scaling**), com o plano conforme está;
- Mudar para a *frame* 24 (a **Frame Rate** é de 24 *fps*);
- Alterar o parâmetro X da escala do plano para 5;
- Marcar a *keyframe* (**Tecla I**, seguida da opção **Scaling**);
- Mudar para a *frame* 48, na janela *Timeline*;
- Alterar o parâmetro X da escala do plano para 1;
- Marcar a *keyframe* (**Tecla I**, seguida da opção **Scaling**);
- Mudar o *layout* para **Game Logic**;
- Com o plano selecionado, adicionar um atuador **Action**;

- Colocar na área logo abaixo do **Play** o nome da animação feita;
- Pôr o campo **Start Frame** a 1 e o campo **End Frame** a 48;
- Ligar o sensor **Mouse Over** a este atuador;
- Verificar o resultado (**Tecla P**), quando apenas o cursor do rato está sobre o plano.

9. Criação e utilização de *scripts*

- Na maioria dos casos, quando se implementa um jogo tem que se programar certas operações. No BGE isso pode fazer-se através de **scripts**, que utilizam a linguagem **Python**. Um exemplo do seu uso pode ser aplicado à necessidade de passar informação entre cenários. Para isso, fazer o seguinte:

- Criar uma nova cena chamada "**Final**" e mudar para **Blender Game**;
- Adicionar uma câmara na posição (1, -1, 1.25), rodada (60°, 0°, 35°);
- Na vista da câmara, criar o texto "**0**", alinhado com a vista da camara;
- Com o texto selecionado, adicionar um sensor **Always**;
- No painel **Properties**, do editor lógico, pressionar o botão **Add Game Property** e alterar o nome da propriedade para "**Points**";
- Colocar o tipo da propriedade como **Integer** e pressionar no botão ;
- No painel **Properties**, do editor lógico, pressionar o botão **Add Text Game Property** e alterar o tipo de texto para **Integer**;
- Adicionar um atuador **Property**, mudar o campo **Mode** para **Add** e pôr no campo **Property** a propriedade **Text** e no **Value** a propriedade **Points**;
- Ligar o sensor **Always** criado anteriormente ao atuador **Property**;
- Na janela **Text Editor**, premir o botão **Open** e escolher o ficheiro "**BGE_LoadInfo.txt**";
- A explicação do código do *script* é a seguinte:


```
import bge                                     (para se poder usar a biblioteca bge)
cont=bge.logic.getCurrentController()          (obtém informações do jogo)
own=cont.owner                                (obtém informações do objeto que usará o script)
props = bge.logic.player_props                (traz da memória a estrutura props)
own['Points']=props['Pts']                     (atribui à propriedade Points o valor de Pts)
```
- Com o texto selecionado, adicionar um controlador do tipo **Python** e no campo em frente à palavra **Script** colocar "**BGE_LoadInfo.txt**";
- Ligar o sensor **Always** ao controlador **Python**;
- Voltar à cena "**Scene**" e selecionar o plano "**Test**" e o texto "**Contador**";
- Na janela **Text Editor**, escolher a opção **Text→Open Text Block** e selecionar o ficheiro "**BGE_SaveInfo.txt**";
- Adicionar um controlador do tipo **Python** ao elemento "**Contador**" e no campo em frente à palavra **Script** colocar "**BGE_SaveInfo.txt**";
- A explicação do código do *script* é a seguinte:


```
import bge                                     (para se poder usar a biblioteca bge)
cont=bge.logic.getCurrentController()          (obtém informações do jogo)
own=cont.owner                                (obtém informações do objeto que usará o script)
props = {}                                    (Inicializa a estrutura props)
props['Pts'] = own["Text"]                     (põe em Pts o valor da propriedade Text)
bge.logic.player_props=props                  (guarda a estrutura props na memória)
```
- Ligar o sensor **Property** (**vida = 0**) ao controlador **Python**;

- Alterar o atuador **Game** para **Scene**, pôr o campo **Mode** a **Set Scene** e colocar no campo **Scene** o cenário **Final**;
- Jogar a partir da cena “MenuIntro” (**Tecla P**) e ver que quando o jogador perde, o cenário muda e são mostrados os pontos que o jogador fez.

10. Criação do executável

- Para se poder criar o executável do jogo, tem que se ter ativa a exportação do projeto para executável. Caso isso não se verifique, tem que se proceder da seguinte forma:

- Na janela **Info**, escolher a opção **File→User Preferences**;
- Selecionar o painel **Addons** e pressionar o botão **Game Engine**;
- Ativar o **addon Game Engine: Save As Game Engine Runtime**;
- Fechar a janela **User Preferences**.

- Tendo então ativa a opção anteriormente referida, para se criar o executável faz-se o seguinte:

- Colocar o jogo na cena “MenuIntro” (a cena onde se fizer o processo abaixo será a cena que aparecerá quando o executável for corrido);
- Na janela **Info**, ir à opção **File→Export→Save as Game Engine Runtime**;
- Escolher a pasta onde criar o executável e o nome desse executável;
- Pressionar o botão **Save as Game Engine Runtime**;
- Ver se na pasta especificada foi criado o executável. Se o jogo tiver sons, esses ficheiros de som deverão ser copiados para junto do executável.

11. Exercício

- Terminar o jogo anterior, criando a possibilidade de no cenário final poder recommear o jogo pressionando o botão esquerdo do rato, ou terminar o jogo pressionando o botão direito do rato.

Como mover um peão num tabuleiro

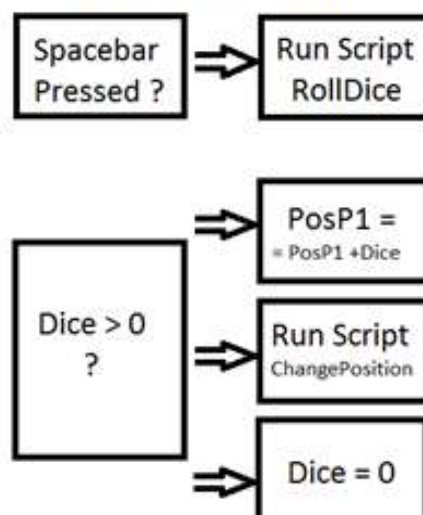


Figura 2: Módulos do jogo a programar.

- Num jogo de tabuleiro, para se poder mover o peão um número de casas igual ao valor que saiu num dado, lançado quando se pressiona a **Tecla Space**, tem

que se programar alguns módulos (conforme se pode ver na figura 2). Uma das formas de se fazer é seguindo o seguinte tutorial:

- Abrir o ficheiro “*FCG_05_BGE_Exercicio_Inicio.blend*”;
- Selecionar o tabuleiro e no painel **Navigation Mesh**, ícone **Scene** (da janela de propriedades), pressionar o botão **Build Navigation Mesh**. Desta forma o peão nunca sairá para fora do tabuleiro;
- Para que o movimento do peão no tabuleiro seja suave, criar um elemento **empty** (*add→Empty→Plain Axes*) na posição (0.0, 10.0, 0.5);
- Com o peão (o cone) selecionado, adicionar um sensor **Always**;
- Pressionar o botão **Add Actuators** e escolher o atuador **Steering**;
- Colocar o campo **Behavior** com **Path Following**;
- Pôr o campo **TargetObject** com o nome do elemento *empty*;
- Pôr no campo **Navigation Mesh** o nome da malha de navegação criada;
- Pôr o campo **Dist** com o valor de 0 (ir para a posição exata do *Empty*);
- Ativar o botão **Visualize** (para ver a linha do percurso a fazer);
- Ligar o sensor ao atuador;
- Selecionar o elemento *Empty* e no painel **Properties**, do editor lógico, pressionar o botão **Add Game Property**;
- Alterar o nome da propriedade para “*Dice*”;
- Colocar o tipo da propriedade como **Integer**;
- Pressionar no botão  (para permitir que depois se possa ver o comportamento desta propriedade no modo de *debug*);
- Na janela **Info**, escolher a opção **Game** e ativar o campo **Show Debug Properties**;

- Uma função muito usada em jogos é a da atribuição de um valor aleatório a uma variável (por exemplo, para simular um dado). Como o BGE não tem blocos pré-definidos que façam isto, tem que se recorrer à programação de um *script*. Assim,

- Na janela **Text Editor**, premir o botão **Open** e selecionar o ficheiro “*BGE_Exercicio_JogarDado.txt*”;
- A explicação do código do *script* é a seguinte:

```
import random          (para se poder usar a biblioteca random)
from bge import logic   (para se poder usar o script nos blocos BGE)
cont=logic.getCurrentController() (obter informações o jogo)
own=cont.owner          (obter informações do objeto que usará o script)
rand=random.randint(1, 2) (Gerar valor aleatório entre 1 e 2)
own['Dado']=rand        (pôr o valor aleatório na propriedade Dado)
```
- Alterar o nome do *script* para “*RollDice*”;
- Com o *Empty* selecionado, adicionar um sensor **Keyboard**;
- Pressionar o **botão esquerdo do rato** sobre o campo vazio situado em frente da área **Key** e, depois, premir a **Tecla SPACE**;
- Ativar o envio de apenas uma resposta positiva ao controlador, mesmo que se mantenha a tecla premida, pressionando a área **Tap**;
- Adicionar um controlador do tipo **Python** e no campo em frente à palavra **Script** colocar “*RollDice*”;

- Ligar os blocos e ver (**Tecla P**) que quando se prime a tecla *space* a propriedade “*Dice*” vai tendo os valores 1 ou 2.

- Uma das maneiras de colocar o peão a andar sobre as casas do tabuleiro é feita da seguinte forma:

- Selecionar o elemento *Empty*;
- No painel **Properties**, do editor lógico, pressionar o botão **Add Game Property**;
- Alterar o nome da propriedade para “*PosP1*”;
- Alterar o tipo da propriedade para **Integer** e premir no botão ;
- Na janela **Text Editor**, ir à opção **Text→Open Text Block**;
- Escolher o ficheiro “*BGE_Exercicio_MudarPos.txt*”;
- A explicação do código do *script* é a seguinte:

```
from bge import logic (para se poder usar o script nos blocos BGE)
cont=logic.getCurrentController() (obter informações o jogo)
own=cont.owner (obter informações do objeto que usará o script)
if own['PosP1']<=6: (Como o peão vai andar nas casas iniciais)
    own.position = [0.0, -own['PosP1']*2.0+10.0, 0.5]
elif own['PosP1']>=7: (Como o peão vai andar nas casas finais)
    own.position = [-(own['PosP1']-6)*2.0, -2.0, 0.5]
```
- Alterar o nome do *script* para “*ChangePosition*”;
- Com o *Empty* selecionado, adicionar um sensor **Property**;
- Colocar **Greater Than** no campo **Evaluation Type**;
- Colocar no campo **Property** a propriedade “*Dice*”;
- Colocar no campo **Value** o valor 0 (vai ativar uma resposta positiva quando a propriedade “*Dice*” for maior que 0);
- Adicionar um controlador do tipo **Python** e no campo em frente à palavra **Script** colocar “*ChangePosition*”;
- Ligar os blocos;
- Com o *Empty* selecionado, adicionar um controlador **And** e ligá-lo ao sensor **Property** (*Dice*>0);
- Adicionar um atuador **Property**;
- Mudar o campo **Mode** para **Add**;
- Colocar no campo **Property** a propriedade “*PosP1*”;
- Colocar no campo **Value** o valor *Dice* (vai adicionar o valor da propriedade “*Dice*” ao valor que a propriedade “*PosP1*” tem);
- Ligar o atuador criado ao último controlador **And** adicionado;
- Adicionar um novo atuador **Property**;
- Deixar o campo **Mode** em **Assign**;
- Colocar no campo **Property** a propriedade “*Dice*” e no campo **Value** o valor 0 (vai atribuir esse valor à propriedade “*Dice*”);
- Ligar o atuador criado ao controlador **And** onde se ligou o atuador anterior (nesta situação, a ordem de execução seguida é que a property ligada a *PosP1* seja processada antes da do *Dice*);
- Verificar (**Tecla P**) que quando se pressiona a **Tecla SPACE** o peão vai andando sobre o tabuleiro;

- É importante referir que as fórmulas usadas para se fazer o movimento do peão dependem do formato do tabuleiro;

- É essencial dizer que a ordem com que os blocos são colocados é importante. Neste exemplo, se o bloco que coloca a zero a propriedade “Dice” estivesse antes do bloco que adiciona o valor da propriedade “Dice” a “PosP1”, o peão não se movia.
- Durante um jogo de tabuleiro, costumam existir casas onde se tem que recuar, avançar, responder a uma pergunta, recomeçar/terminar o jogo ou fazer outra qualquer ação. Assim, para se ter, por exemplo, a situação de um recuo de três (3) casas quando o peão calha na casa amarela do tabuleiro, pode-se fazer o seguinte:

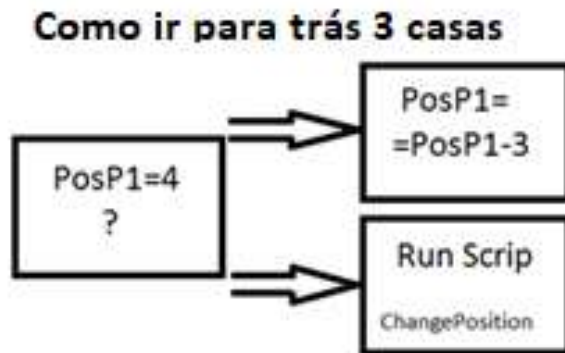


Figura 3: Módulo de recuo a programar.

- Com o *Empty* selecionado, adicionar um sensor **Property**;
- Colocar no campo **Property** a propriedade “PosP1”;
- Colocar no campo **Value** o valor 4;
- Ligar esse sensor ao bloco **Python** com script “ChangePosition”;
- Adicionar um atuador **Property**;
- Mudar o campo **Mode** para **Add**;
- Colocar no campo **Property** a propriedade “PosP1”;
- Colocar no campo **Value** o valor -3;
- Ligar o último sensor criado ao atuador;
- Ver (**Tecla P**) que o peão volta para trás quando calha na casa amarela.